

Project Executable Files

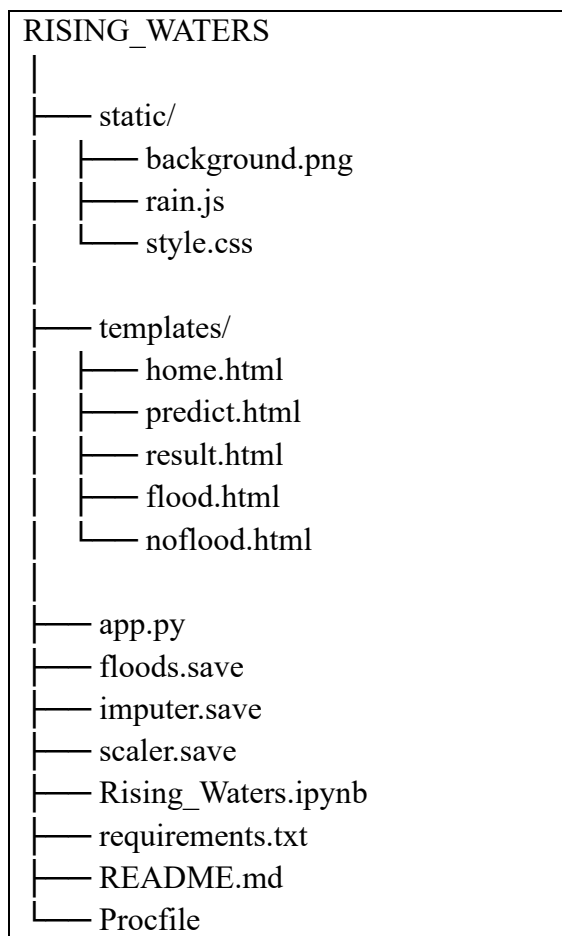
Date	1 July 2026
Team ID	SWTID-2026-8096
Project Name	Rising Waters
Maximum Marks	3 Marks

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt dependency file	Yes
4	Database schema / seed files	NA
5	Environment configuration file (.env.example)	No
6	Deployed application URL	Yes
7	APK / Executable binary	Yes
8	Docker file / Containerization configuration	No
9	Test files and test results	Yes
10	Demo video or walkthrough	Yes

Step 2: File / Folder Structure:

The Rising Waters project follows a well-organized and modular folder structure. The static folder contains the CSS, JavaScript, and image files used for the user interface, while the templates folder stores all HTML pages. The app.py file serves as the main Flask application, integrating the trained machine learning model (floods.save) along with preprocessing files (imputer.save and scaler.save) to perform flood prediction. Supporting files such as requirements.txt, README.md, and Procfile provide dependency management, project documentation, and deployment configuration, ensuring the application is easy to maintain and deploy.



Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Rising Waters Home
Login Credentials (Demo)	Not Required
Platform /Hosting Provider	Render
Repository Link	Jagadeesh-krishna/Flood-Prediction-
Demo Video Link	https://youtu.be/Yfx8tsqdurE?si=4rn5FYQKzj-2xWT0

Step 4: Run Instructions

1. Clone or download the project from the GitHub repository.
2. Open the project folder in **Visual Studio Code**.
3. Install the required dependencies:

```
pip install -r requirements.txt
```
4. Start the Flask application:

```
python app.py
```
5. Open a web browser and visit:
<http://127.0.0.1:5000>
6. Enter the annual and seasonal rainfall values.
7. Click **Predict** to view the flood prediction result.
8. Alternatively, access the deployed application using the Render URL.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Prediction accuracy depends on the quality of rainfall input data.	Use valid rainfall values collected from reliable sources.
2	Real-time weather and rainfall APIs are not integrated.	Planned as a future enhancement.
3	User login and prediction history features are not available.	Can be implemented in future versions of the application.